

Year:-	Academic Year- 2026-27	Semester:- V
Experiment # 7		

Aim:

Conduct a study of Deadlock in Operating system and perform following tasks

1. Identify the necessary conditions for Deadlock
2. Identify the techniques for avoiding deadlock
3. Conduct a study for Bankers Algorithm and Implementation of Banker's Algorithm using any programming language.
4. Plot safe/unsafe state transitions using matplotlib or networkx

Prerequisite:

- Understanding of **process synchronization**, **resource allocation**, and **critical section problems**
- Familiarity with concepts: Deadlock, Safe/Unsafe states, Resource Allocation Graphs
- Basic programming skills (e.g., Python or C)
- Introductory knowledge of data visualization libraries like `matplotlib` or `networkx`

Theory:

A **deadlock** is a situation where a set of processes is blocked because each process is holding a resource and waiting for another one.

The **Banker's Algorithm**, proposed by Dijkstra, is a resource allocation and deadlock avoidance algorithm used to check whether a safe state exists before allocating resources.

Deadlock Concepts

Necessary Conditions for Deadlock (Coffman's conditions):

1. **Mutual Exclusion** – At least one resource must be held in a non-shareable mode.
2. **Hold and Wait** – A process is holding at least one resource and waiting to acquire others.
3. **No Preemption** – Resources cannot be forcibly taken from processes.
4. **Circular Wait** – A set of processes are waiting for each other in a circular chain.

Deadlock Avoidance Techniques:

- Use of **Banker's Algorithm** to ensure system stays in a safe state
- **Request-before-hold** strategies
- **Resource ordering**
- **Preemption policies**

Methodology:

Task 1: Study Deadlock Conditions

- Document and explain the **four necessary conditions** for deadlock.
- Provide real-world examples of where these conditions may exist.

Year:-	Academic Year- 2026-27	Semester:- V
--------	------------------------	--------------

Task 2: Identify Deadlock Avoidance Techniques

- Discuss and compare:
 - **Deadlock Prevention** vs. **Deadlock Avoidance**
 - Use cases and pros/cons of **Banker's Algorithm**
 - Conditions under which preemption can help

Task 3: Implement Banker's Algorithm

- Take input:
 - **Available resources**, **Max demand**, and **Allocated resources** for n processes and m resource types
- Compute:
 - **Need = Max - Allocation**
- Determine:
 - Whether the system is in a **safe state**
 - Generate a **safe sequence** (if one exists)

Implementation can be done in Python, C, or any preferred language.

Task 4: Visualize Safe/Unsafe State Transitions

- Use **matplotlib** or **networkx** in Python to:
 - Plot the transitions between states
 - Mark **safe** and **unsafe** states with different colors or node styles
 - Optionally simulate process resource requests and show how state changes

Reference Link:

<https://banker-s-algorithm-simulator.vercel.app/>

Outcome:

Upon successful completion of this lab, students will be able to:

- Understand the concept of deadlock and its four necessary conditions
- Differentiate between prevention, avoidance, and detection strategies
- Implement Banker's Algorithm to determine safe states
- Visualize system state transitions through graphs or charts

Questions for Assessment (QA):

- What are the four necessary conditions for deadlock? Can a system avoid deadlock by eliminating just one?
- Explain the difference between safe and unsafe states.
- How does Banker's Algorithm prevent deadlock?
- What happens if a resource request cannot be granted in the current state?
- How can graphs be used to model and analyze system states?
- Can the Banker's Algorithm be applied in real-time systems? Why or why not?

Year:-	Academic Year- 2026-27	Semester:- V
---------------	-------------------------------	---------------------

PART B

(TO BE COMPLETED BY STUDENTS)

(Students must submit the soft copy as per the following segments within the stipulated deadline. The soft copy must be uploaded on the portal or emailed to the concerned Lab in charge of Faculties at the end of the practical; in case the portal is not accessible)

Roll No: A056	Name: P.CHARAN
Class: CSDS-A	Batch: A2
Date of Experiment	Date of Submission

Study / Implementation details:

Task 1:

Deadlock is a situation where processes wait for each other forever.

Four Conditions

1. **Mutual Exclusion** – Only one process can use a resource at a time.
Example: Only one person can use a printer.
2. **Hold and Wait** – A process holds one resource and waits for another.
Example: A person holds one key and waits for another key.
3. **No Preemption** – A resource cannot be forcefully taken from a process.
Example: A printer cannot be taken while it is printing.
4. **Circular Wait** – Processes wait for resources held by each other in a circle.
Example: P1 waits for P2, P2 waits for P3, and P3 waits for P1.

All four conditions together can cause deadlock.

Year:-	Academic Year- 2026-27	Semester:- V
---------------	-------------------------------	---------------------

Task 2:

Prevention vs Avoidance

Prevention

Stops deadlock by removing one condition. Checks before allocating resources.

Simple method.

Example: Do not allow Hold and Wait.

Avoidance

More complex.

Example: Banker's Algorithm.

Banker's Algorithm

Banker's Algorithm checks whether giving resources to a process will keep the system **safe**.

Steps:

- Calculate Need = Max - Allocation
- Check available resources.
- Find a **safe sequence**.
- If all processes can finish → **Safe State**.
- Otherwise → **Unsafe State**.

Advantages:

- Prevents deadlock.
- Finds safe sequence.
- Works with multiple resources.

Disadvantages:

- Maximum demand must be known.
- Requires more calculations.
- Processes may have to wait.

Preemption

Preemption means taking a resource from one process and giving it to another.

Year:-	Academic Year- 2026-27	Semester:- V
---------------	-------------------------------	---------------------

It can help when:

- The resource can be safely taken.
- The process can continue later.

Example: Taking a CPU from one process and giving it to another.

Task 3 & 4:

Code:

```
import numpy as np
import matplotlib.pyplot as plt
import networkx as nx

# -----
# BANKER'S ALGORITHM
# -----

# Number of processes and resources

n = int(input("Enter number of processes: "))
m = int(input("Enter number of resource types: "))

# Input Available resources

print("\nEnter Available resources:")
available = list(map(int, input().split()))

# Input Maximum demand

print("\nEnter Max demand matrix:")
```

Year:-

Academic Year- 2026-27

Semester:- V

```
max_demand = []
```

```
for i in range(n):
```

```
    row = list(map(int, input(f'P {i}: ').split()))
```

```
    max_demand.append(row)
```

```
# Input Allocation
```

```
print("\nEnter Allocation matrix:")
```

```
allocation = []
```

```
for i in range(n):
```

```
    row = list(map(int, input(f'P {i}: ').split()))
```

```
    allocation.append(row)
```

```
# Convert to NumPy arrays
```

```
available = np.array(available)
```

```
max_demand = np.array(max_demand)
```

```
allocation = np.array(allocation)
```

```
# -----
```

```
# Calculate Need
```

```
# Need = Max - Allocation
```

```
# -----
```

```
need = max_demand - allocation
```

Year:-

Academic Year- 2026-27

Semester:- V

```
print("\n===== BANKER'S ALGORITHM =====")
```

```
print("\nMax Demand:")
```

```
print(max_demand)
```

```
print("\nAllocation:")
```

```
print(allocation)
```

```
print("\nNeed = Max - Allocation:")
```

```
print(need)
```

```
print("\nAvailable:")
```

```
print(available)
```

```
# -----
```

```
# Safety Algorithm
```

```
# -----
```

```
work = available.copy()
```

```
finish = [False] * n
```

```
safe_sequence = []
```

```
while len(safe_sequence) < n:
```

Year:-

Academic Year- 2026-27

Semester:- V

found = False

for i in range(n):

Check if process can finish

if not finish[i] and np.all(need[i] <= work):

Process finishes and releases resources

work = work + allocation[i]

finish[i] = True

safe_sequence.append(i)

found = True

print(f"\nP {i} can finish.")

print("Work =", work)

No process can finish

if not found:

break

Result

Year:-

Academic Year- 2026-27

Semester:- V

```
if len(safe_sequence) == n:
```

```
    print("\nSystem is in a SAFE STATE.")
```

```
    print("Safe Sequence:")
```

```
    print(" -> ".join("P" + str(i) for i in safe_sequence))
```

```
else:
```

```
    print("\nSystem is in an UNSAFE STATE.")
```

```
    print("No safe sequence exists.")
```

```
# -----
```

```
# VISUALIZATION
```

```
# -----
```

```
G = nx.DiGraph()
```

```
# Add process nodes
```

```
for i in range(n):
```

```
    G.add_node("P" + str(i))
```

```
# Add edges according to safe sequence
```

Year:-	Academic Year- 2026-27	Semester:- V
---------------	-------------------------------	---------------------

```
if len(safe_sequence) > 1:
```

```
    for i in range(len(safe_sequence) - 1):
```

```
        p1 = "P" + str(safe_sequence[i])
```

```
        p2 = "P" + str(safe_sequence[i + 1])
```

```
        G.add_edge(p1, p2)
```

```
# Draw graph
```

```
plt.figure(figsize=(10, 5))
```

```
if len(safe_sequence) == n:
```

```
    node_colors = ["green"] * n
```

```
    title = "Banker's Algorithm - SAFE State"
```

```
else:
```

```
    node_colors = ["red"] * n
```

```
    title = "Banker's Algorithm - UNSAFE State"
```

```
pos = nx.spring_layout(G, seed=42)
```

```
nx.draw(
```

Year:-	Academic Year- 2026-27	Semester:- V
---------------	-------------------------------	---------------------

```
G,
pos,
with_labels=True,
node_color=node_colors,
node_size=2500,
font_size=12,
font_weight="bold",
arrows=True
)
```

```
plt.title(title)
```

```
plt.show()
```

Output:

